CSED232 Principles of Software Construction

Object-Oriented Contracts

Kyungmin Bae

Department of Computer Science and Engineering
POSTECH

Spring 2026

Recall: Specifications

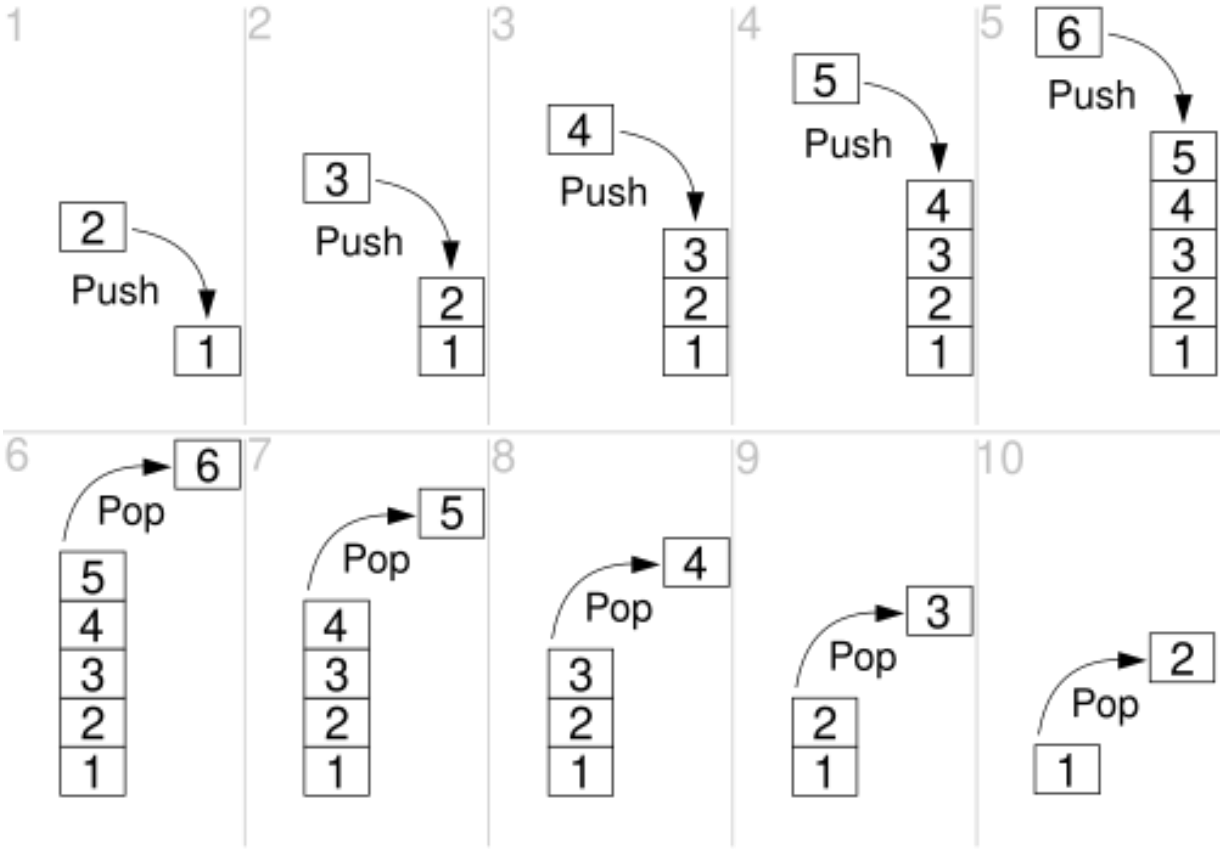
- What are we supposed to build?
  - modules, classes, objects, types, functions, …
- A contract between the user and the manufacturer of the product
  - describes their expectations of each other
- Facilitates change
  - code changes, but specification may not

Class Specifications

Recall: Stack

- A last-in, first-out (LIFO) data structure
  - elements are added and removed at the `top`

- Basic operations
  - `push(x)`: add element x to the top
  - `pop()`: remove and return the top element
  - `size()`: return the number of elements
  - `capacity()`: return the maximum number of elements (for bounded stacks)



What is a Specification of Stack?

```
public interface Stack {
    void push(Object item);
    Object pop();
    int size();
    int capacity();
    ...
}
```

- Specifications of each method: preconditions and postconditions
- But `Stack` objects have internal states!
  - How do we specify the behavior of stateful objects?

Many Implementations of Stack

- There are many possible implementations, using
  - an array
  - a linked list
  - …
- Different implementations have different internal representations
  - specifications should not depend on a specific internal representation
- How do we specify behavior independently of implementation?

Data Abstraction

- Give clients only operations, not data
  - operations are the specification
- Separate what from how
  - hide internal representation
- Abstract Data Type (ADT)
  - fundamental concept in computer science

Barbara Liskov
(Turing Award, 2008)

Abstract View of Objects

- Objects are viewed by their abstract values
  - internal representation is ignored (information hiding)
- Characterized by a set of operations
  - the only way clients interact with the object
- Example: Stack
  - abstract view: a mathematical list of elements

Specification of Stack

```
//@ requires:
//@    true
//@ ensures:
//@    returns the length of the stack

int size();
```

```
//@ requires:
//@    item is not null and size() < capacity()
//@ ensures:
//@    modifies the stack by adding item to the end of the stack

void push(Object item);
```

```
//@ requires:
//@    size() > 0
//@ ensures:
//@    modifies the stack by removing the last item, and returns the removed item

Object pop();
```

Types of Operations

- Observers (or getters)
  - return information about the object without modifying it
  - e.g., `size()`, `capacity()`, …
- Mutators (or setters)
  - modify the object's state
  - e.g., `push(item)`, `pop()`, …
- Producers
  - create new objects without modifying the existing one
  - e.g., `clone()`, …

Are All Abstract Values Allowed?

- For Stack, the following condition must always hold:

$$0 <= size() <= capacity()$$

- Class invariant
  - constraints that must be preserved by all operations

- Example

```
public interface Stack {
//@ invariant: 0 ≤ size() ≤ capacity()

    ...

}
```

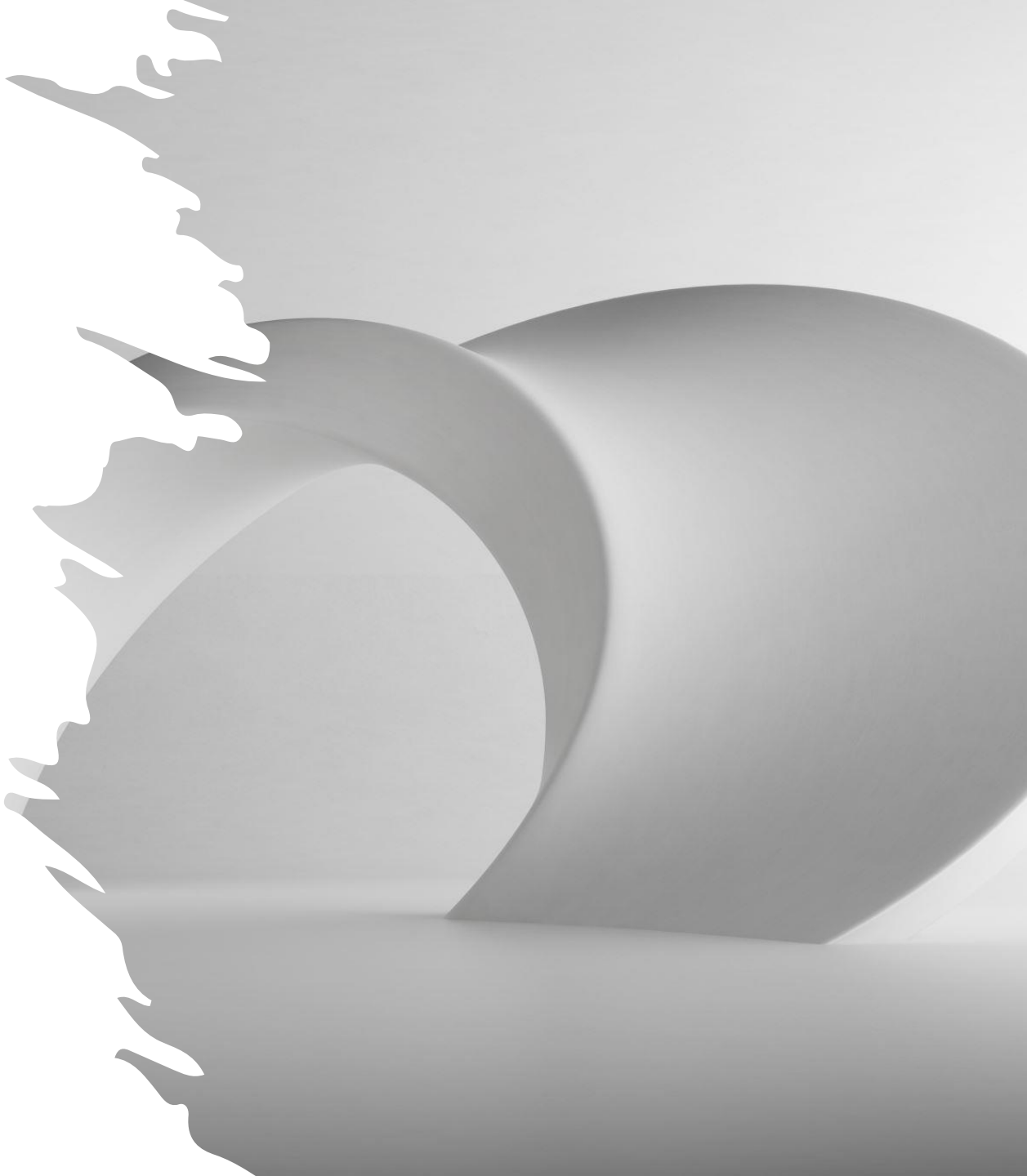
Example: What is the Class Invariant?

```
public interface ValueRange {

    //@ ensures: returns the first entry of the pair
    int getLower();

    //@ ensures: returns the second entry of the pair
    int getUpper();


    //@ requires : i < getUpper()
    //@ ensures : modify the first entry to i
    void setLower(int i);

    //@ requires : i > getLower()
    //@ ensures : modify the second entry to i
    void setUpper(int i);
}
```

- Abstract value of ValueRange: a pair (l, u) of integers

Implementations and Specifications

- Abstract objects
  - specifications with respect to abstract values
  - how clients should think about the object
- Concrete objects
  - internal specifications with respect to internal concrete values
- Abstraction
  - defines how concrete objects are interpreted as abstract values
- Representation invariant
  - class invariant of concrete implementation

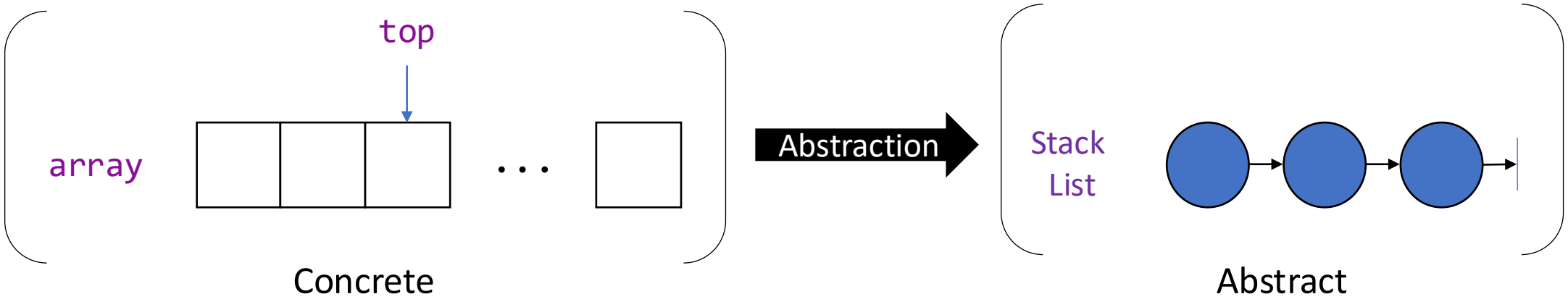


Example (1)

```
class ArrayStack implements Stack {
    private Object[] array;
    private int top = 0;

    ArrayStack(int capacity) {
        array = new Object[capacity];
    }


    @Override
    public void push(Object item) {
        array[top++] = item;
    }

    @Override
    public Object pop() {
        return array[--top];
    }

    ...
}
```

Example (2)

- Abstraction from `ArrayStack` to `Stack`



- Maps concrete values into abstract values

Example: CharSet

```
/**
 * A finite mutable set of Characters
 */
class CharSet {

    // @ensures: add c into the set
    public void insert(Character c) { ... }

    // @ensures : remove c from the set
    public void delete(Character c) { ... }

    // @requires : true
    // @ensures : returns true iff the set includes c
    public boolean member(Character c) { ... }
...
}
```

An Implementation: Is It Correct?

```
class CharSet {
    private List<Character> elmList = new ArrayList<Character>();

    public void insert(Character c) {
        elmList.add(c);
    }

    public void delete(Character c) {
        elmList.remove(c);
    }

    public boolean member(Character c) {
        return elmList.contains(c)
    }
...
}
```

- Where is the defect?

Example: Representation Invariant of CharSet

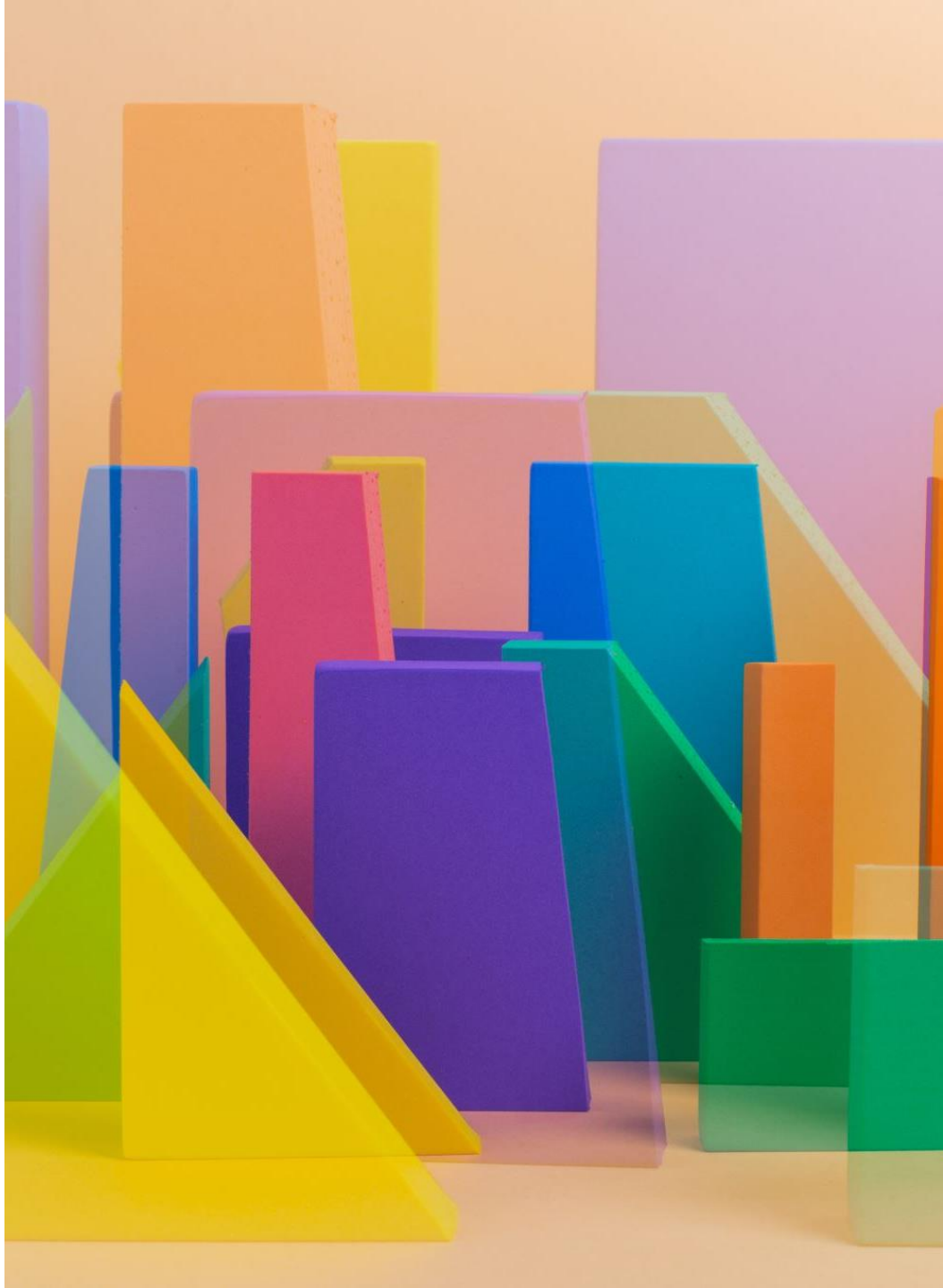
```
class CharSet {
    //@ invariant: elmList has no nulls and no duplicates

    private List<Character> elmList = new ArrayList<Character>();

    public void insert(Character c) {
        elmList.add(c);
    }

    public void delete(Character c) {
        elmList.remove(c);
    }

    public boolean member(Character c) {
        return elmList.contains(c)
    }
...
}
```

- Now we can locate the error

Abstraction and Representation Exposure

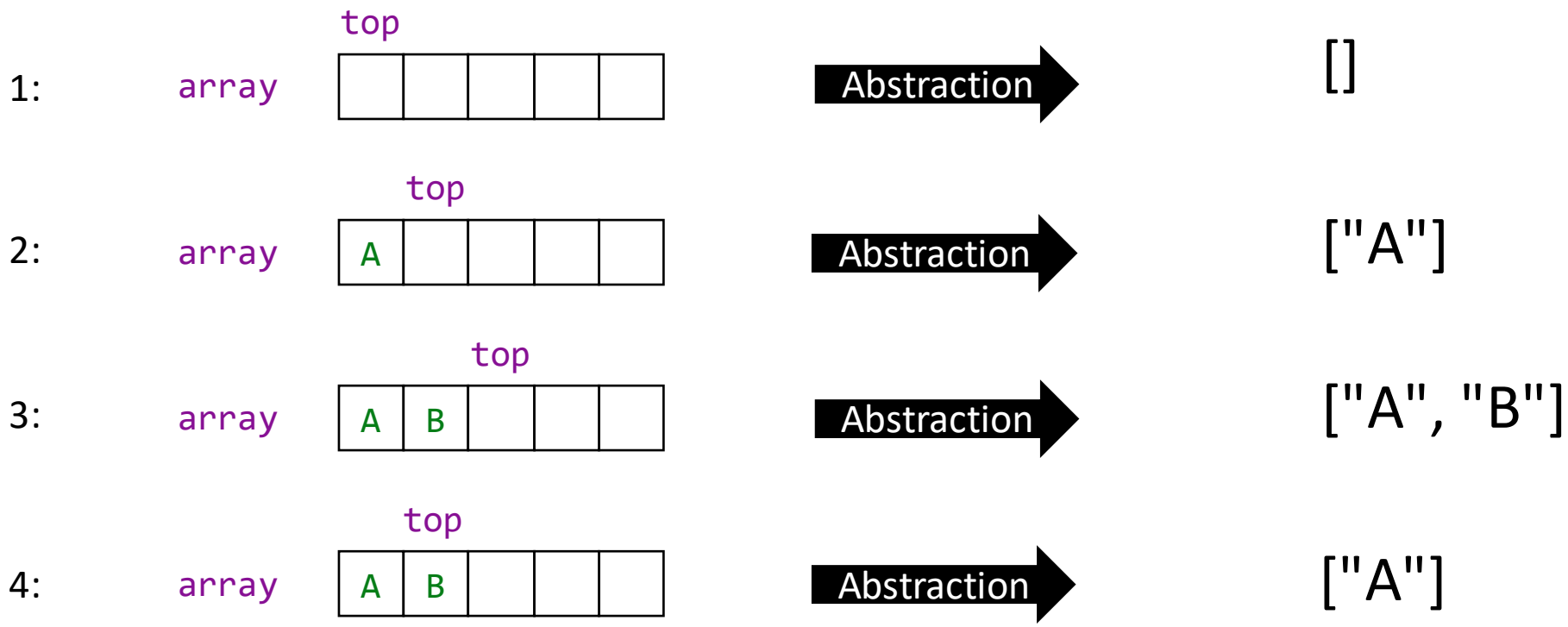
Properties of Abstraction

- Abstraction is a function
  - Any concrete object maps to an abstract object
- Abstraction is onto
  - For any abstract object, there exists a corresponding concrete object
- Abstraction is not one-to-one
  - Two concrete objects may map to the same abstract object



Example: ArrayStack

```
ArrayStack stack = new ArrayStack(5);
stack.push("A");
stack.push("B");
stack.pop();
```



Example: `CharSet`

- Different implementation of `member`:

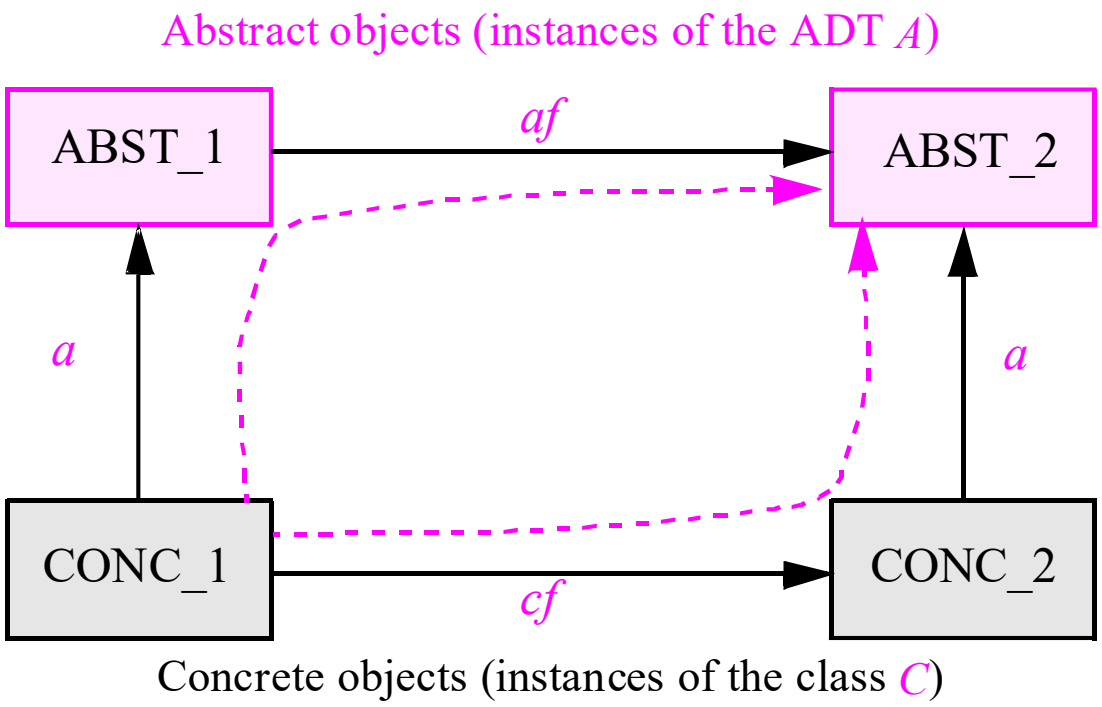
```
boolean member(Character c1) {
    var i = elmList.indexOf(c1);
    if (i == -1)
        return false;
    elmList.add(0, elmList.remove(i)); // move-to-front optimization
    return true;
}
```

- Mutates concrete object, but does not change the abstract object
  - Abstraction maps both representations to the same abstract object.
- Such mutation is not observable by the client!

Implementation Meets Specification

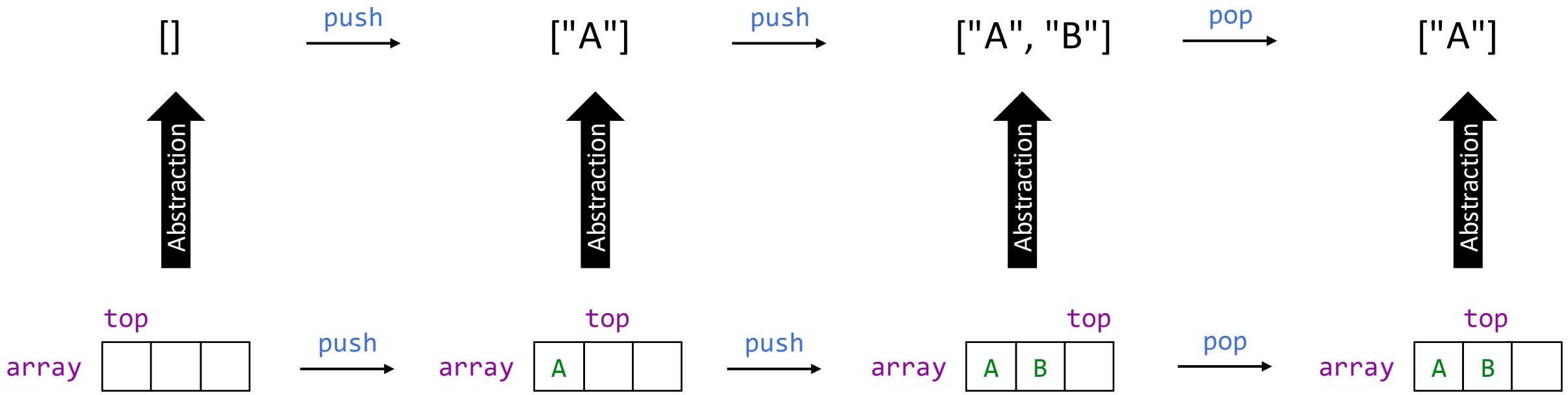
- Method f
- Abstraction a
- Concrete f satisfies abstract f

  a ; af = cf ; a



Example: ArrayStack

```
ArrayStack stack = new ArrayStack(3);
stack.push("A");
stack.push("B");
stack.pop();
```



Example: CharSet

```
CharSet cset = new CharSet();
cset.insert('A');
cset.insert('B’);
cset.insert('C');
cset.member('B');    // move-to-front optimization version
```

Representation Exposure

- Direct access to internal representation (can happen for private field)

```
class ArrayStack implements Stack {
    ...

    Object[] getElements() { return array; }

    ...
```

- Class invariant can be violated even if specification is correct

```
ArrayStack stack = new ArrayStack(10);
stack.push(1);
stack.getElements()[0] = null;
System.out.println(stack.pop());
```

Avoiding Representation Exposure (1)

- Declaring `private`
  - Not enough
  - Methods may return references to internal representation

- Example
  - The `getElements` method of `ArrayStack` in the previous slide

Avoiding Representation Exposure (2)

- The keyword `final`
  - Indicates that the variable can be assigned only once
  - Example

```
final int x = 1;
final StringBuilder y = new StringBuilder();

x = 2;                          // compile-time error
y = new StringBuilder(100);     // compile-time error

y.append('a');                  // allowed: modifying the internal state of the object y
System.out.println(y);          // output: a
```

- Declaring `final`
  - Prevents modification of references and primitive values
  - Still not enough, as method calls can change internal representation

Avoiding Representation Exposure (3)

- Immutable objects
  - Objects whose state cannot be modified after it is created
- Requires different designs
  - Operations return new objects rather than changing existing ones
- Benefits
  - No representation exposure, and representation invariants cannot be broken
  - Safe sharing, thread-safe, simpler reasoning, …
- Immutable classes in Java libraries
  - String, Character, Integer, Double, …

Avoiding Representation Exposure (4)

- Immutable containers
  - List.of(…), Set.of(…), Map.of(…), …
- Immutable wrappers
  - Collections.unmodifiableList(…), Collections.unmodifiableSet(…), …
- Example

```
class ArrayStack implements Stack {
    ...
    List getElements() { return List.of(array); }
    ...
```

- Use Records!

Records and Pattern Matching

Java Records

- A syntactic sugar of class
  - greatly simplifies the equivalent class declaration for immutable types
  - fields are described in a header
  - constructor and accessor/getters are declared automatically

- Example

```
record Point(double x, double y) {}
```

  - an immutable class with two private fields: x and y
  - a public accessor for each fields: x() and y()
  - has a constructor to initialize these two fields
  - toString(), equals(), and hashCode() methods are automatically created

Example

```
record Rectangle(double length, double width) {

    public double length() {
        System.out.println("Length is " + length);
        return length;
    }
}
```

```
Rectangle r = new Rectangle(4,5);
System.out.println("length: " + length() + " width: " + width());
```

- You may define your own accessor methods and constructors

Class Pattern Matching (1)

- Example

```
interface Shape { }
record Rectangle(double length, double width) implements Shape { }
record Circle(double radius) implements Shape { }
```

- Pattern matching for `instanceof`

```
public static double getPerimeter(Shape shape) throws IllegalArgumentException {
    if (s instanceof Rectangle r) {
        return 2 * r.length() + 2 * r.width();
    } else if (s instanceof Circle c) {
        return 2 * c.radius() * Math.PI;
    } else {
        throw new IllegalArgumentException("Unrecognized shape");
    }
}
```

Class Pattern Matching (2)

- Pattern matching for `switch`

```
public static double getPerimeter(Shape s) throws IllegalArgumentException {
    return switch (s) {
        case Rectangle r  -> 2 * r.length() + 2 * r.width();
        case Circle c  -> 2 * c.radius() * Math.PI;
        default  -> throw new IllegalArgumentException("Unrecognized shape");
    };
}
```

- `When` clause

```
static void test(Object obj) {
    switch (obj) {
        case String s when s.length() == 1 -> System.out.println("Short: " + s);
        case String s                      -> System.out.println(s);
        default                            -> System.out.println("Not a string");
    }
}
```

Record Patterns (1)

- For records, we can perform pattern matching on its components

```
static void printAngleFromXAxis(Object obj) {
    if (obj instanceof Point(double x, double y)) {
        System.out.println(Math.toDegrees(Math.atan2(y, x)));
    }
}
```

- C.f., when using ordinary class patterns

```
static void printAngleFromXAxisTypePattern(Object obj) {
    if (obj instanceof Point p) {
        System.out.println(Math.toDegrees(Math.atan2(p.y(), p.x())));
    }
}
```

Record Patterns (2)

- We can nest a record pattern inside another record pattern

```
enum Color { RED, GREEN, BLUE }
record ColoredPoint(Point p, Color c) {}
record ColoredRectangle(ColoredPoint upperLeft, ColoredPoint lowerRight) {}

static void printXCoordOfUpperLeftPointWithPatterns(ColoredRectangle r) {
    if (r instanceof ColoredRectangle(
            ColoredPoint(Point(var x, var y), var upperLeftColor),
            var lowerRightCorner)) {
        System.out.println("Upper-left corner: " + x);
    }
}
```

Caveats in Pattern Matching (1)

- Pattern matching for `switch` must be exhaustive

```
record Pair<T>(T x, T y) {}
class A {}
class B extends A {}

static void notExhaustive(Pair<A> p) {
    switch (p) {
        // error: the switch statement does not cover all possible input values
        case Pair<A>(A a, B b) -> System.out.println("Pair<A>(A a, B b)");
        case Pair<A>(B b, A a) -> System.out.println("Pair<A>(B b, A a)");
    }
}
```

- We can use `default` to make it exhaustive

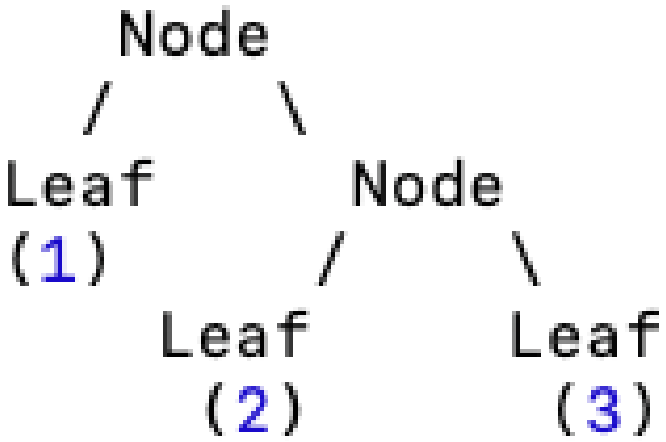
Caveats in Pattern Matching (2)

- We can also use `sealed` classes for exhaustive matching

```
record Pair<T>(T x, T y) {}
sealed interface I permits C, D {}
record C(String s) implements I {}
record D(String s) implements I {}

static void exhaustiveSwitch(Pair<I> p) {
    switch (p) {
        case Pair<I>(I i, C c) -> System.out.println("C = " + c.s());
        case Pair<I>(I i, D d) -> System.out.println("D = " + d.s());
    }
}
```

Example: Binary Tree (1)

- Binary tree
  - A node has two children
  - A leaf node has an integer value



- Java Implementation

```
sealed interface IntTree permits Leaf, Node {}

record Leaf(int value) implements IntTree {}

record Node(IntTree left, IntTree right) implements IntTree {}
```

Example: Binary Tree (2)

- Sum of all nodes

```
static int sum(IntTree tree) {
    return switch (tree) {
        case Leaf(int value) -> value;
        case Node(IntTree left, IntTree right) -> sum(left) + sum(right);
    };
}
```

- Example

```
IntTree tree = new Node(new Leaf(1), new Node(new Leaf(2), new Leaf(3)));
System.out.println("Sum: " + sum(tree));
```

General Contracts for Objects

Object Equality

- Two objects are equal when they
  - have the same value?
  - have the same abstract value?
  - are the same object (same identity/reference)?
  - …

- Defining the precise meaning of equality is not trivial.
  - Indeed, they are several different notion of equality

Expected Properties of Equality

- Reflexive
  - `a.equals(a) == true`
- Symmetric
  - `a.equals(b)` if and only if `b.equals(a)`
- Transitive
  - `a.equals(b)` and `b.equals(c)` implies `a.equals(c)`
- Such a relation is called an equivalence relation

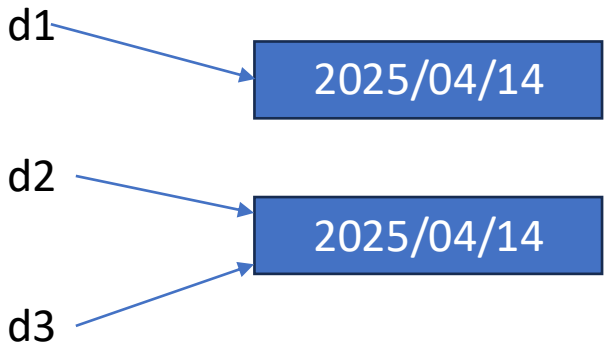
Reference Equality

- Two objects `a` and `b` are reference equivalent iff
  - they refer to the same object (same identity/reference)
  - that is, `a == b` returns `true`
- Reference equality is an equivalence relation
  - reflexive, symmetric, transitive
- Reference equality is the strongest definition of equality
  - the smallest equivalence relation on objects (due to reflexivity)
  - It is sometimes, but not always, what we want

Example: Reference Equality

```
LocalDate d1 = LocalDate.of(2025, 4, 14);
LocalDate d2 = LocalDate.of(2025, 4, 14);
LocalDate d3 = d2;
```

```
System.out.println(d1 == d2);    // output: false
System.out.println(d2 == d3);    // output: true
```



- Sometimes, we want an equivalence relation weaker than ==
  - Java allows classes to override the `equals` method

`Object.equals` Method (1)

- Defines an equivalence relation
  - Except when involving `null`
- Should be consistent
  - but allow results to change if object is mutated
- `a.equals(b)` is asymmetric with `null`:
  - Throws an exception if `a == null`
  - Return `false` if `b == null`

equals

```
public boolean equals(Object obj)
```

Indicates whether some other object is "equal to" this one.

The `equals` method implements an equivalence relation on non-null object references:

- It is *reflexive*: for any non-null reference value `x`, `x.equals(x)` should return `true`.
- It is *symmetric*: for any non-null reference values `x` and `y`, `x.equals(y)` should return `true` if and only if `y.equals(x)` returns `true`.
- It is *transitive*: for any non-null reference values `x`, `y`, and `z`, if `x.equals(y)` returns `true` and `y.equals(z)` returns `true`, then `x.equals(z)` should return `true`.
- It is *consistent*: for any non-null reference values `x` and `y`, multiple invocations of `x.equals(y)` consistently return `true` or consistently return `false`, provided no information used in `equals` comparisons on the objects is modified.
- For any non-null reference value `x`, `x.equals(null)` should return `false`.

`Object.equals` Method (2)

- Implements reference equality

```
public class Object {
    ...

    public boolean equals(Object obj) {
        return (this == obj);
    }

    ...
}
```

- Subclasses can override to implement a different equality
  - provided they meet the specification of `equals`

Behavioral Equivalence

- Two objects a and b are behaviorally equivalent iff
  - no sequence of calls to their methods can distinguish between them.

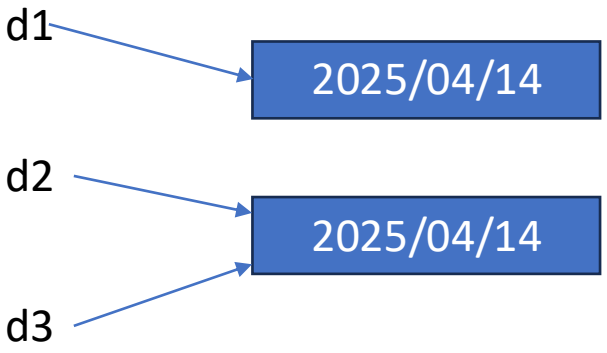
- For immutable types:
  - two objects with the same (abstract) values are behaviorally equivalent
  - immutable types must implement equals themselves

- For mutable types:
  - in most cases, two objects can be distinguished by mutating one of them
  - observational equivalent when they have the same (abstract) values

Example: `LocalDate` Revisited

```
LocalDate d1 = LocalDate.of(2025, 4, 14);
LocalDate d2 = LocalDate.of(2025, 4, 14);
LocalDate d3 = d2;
```

```
System.out.println(d1 == d2);        // output: false
System.out.println(d2 == d3);        // output: true

System.out.println(d1.equals(d2));   // output: true
System.out.println(d1.equals(d3));   // output: true
```



Another Example (1)

```
public class Point {
    private final int x;
    private final int y;

    ...

    @Override
    public boolean equals(Point p) {
        return p.x == x && p.y == y;
    }

    ...
}
```

- It does not override `Object.equals`!
  - It overloads `equals(Point)`, instead of overriding `equals(Object)`

Another Example (2): A Fix

```
public class Point {
    ...

    @Override
    public boolean equals(Object o) {
        if (o instanceof Point p)
            return p.x == x && p.y == y;
        return false;
    }

    ...
}
```

- Notice the use of the `instanceof` operator

Another Example (3): Subclassing

```
class ColorPoint extends Point {
    private final Color color;
    ...

    @Override
    public boolean equals(Object o) {
        if (o instanceof ColorPoint cp)
            return super.equals(o) && cp.color == color;
        return false;
    }

    ...
}
```

- This is not symmetric!

```
Point p = new Point(1, 2);
ColorPoint cp = new ColorPoint(1, 2, RED);


System.out.println(p.equals(cp));  // true
System.out.println(cp.equals(p));  // false
```

Another Example (4): A Potential Fix

```java
class ColorPoint extends Point {
    ...
    @Override
    public boolean equals(Object o) {
        return switch (o) {
            case ColorPoint cp -> super.equals(o) && cp.color == color;
            case Point _ -> super.equals(o);
            default -> false;
        };
    }
    ...
}
```

- Symmetric but not transitive!

```java
ColorPoint p1 = new ColorPoint(1, 2, RED);
Point p2 = new Point(1, 2);
ColorPoint p3 = new ColorPoint(1, 2, BLUE);


System.out.println(p1.equals(p2)); // true
System.out.println(p2.equals(p3)); // true
System.out.println(p1.equals(p3)); // false
```

Subclassing with Overriding `Object.equals`

- Fundamental problem of equivalence in object-oriented languages

  "There is no way to extend an instantiable class and add a value component while preserving the equals contract, unless you're willing to forgo the benefits of object-oriented abstraction." - Effective Java, Item 10

- Some workarounds
  1. Composition over subclassing: make ColorPoint "contain" a Point instance
  2. Use a abstract class: declare both as subclasses of a common abstract class

Another Example (5): A Workaround

```
class ColorPoint {
    private final Point point;
    private final Color color;
    ...

    @Override
    public boolean equals(Object o) {
        if (o instanceof ColorPoint cp)
            return cp.point.equals(point) && cp.color.equals(color);
        else
            return false;
    }

    // Returns the point-view of this color point.
    public Point asPoint() {
        return point;
    }

    ...
}
```

`Object.hashCode`

- Returns a hash code integer for the object
  - used for hash tables (e.g., `java.util.HashMap`)

- Contracts
  - `o.hashCode()` remains the same so long as `o` does not change
  - **If** `a.equals(b)`, **then** `a.hashCode() == b.hashCode()`

- Always override `hashCode` when you override `equals`!

References

- Core Java
  - Chapter 4.7, Chapter 5.8, Chapter 5.9
- Effective Java
  - Chapter 3
- “Program Development in Java” by B. Liskov
  - Chapter 5
- Wikipedia
  - Class invariant

Question?